

JK Tyre PCR Curing - API & Data Flow

Request lifecycle | SAP production feed | DB read/write

PART 1 - SAP Production API (external - we CALL it)

File: sap_production.py - pulls ACTUAL produced quantities from SAP (source for the freezing-plan subtraction).

Method / URL	GET (OData) https://s4api.sap.jktyre.in:44305/.../ZINFI_PLT_API_SRV/InfProdDtSet
Auth / SSL	HTTP Basic (service account INFI_CON) verify=False timeout 300s
Filter	Plant 1300, PFrDt/PToDt (date range), Mtart ZFGS, Matkl 1125, PType PD; \$format=json
Response	data['d']['results'] = list of records
Key fields	Matnr = SKU, Ydate = date, ProdQty = qty produced, MeinhEq = unit

How we read it

```
data = response.json()
results = data.get('d', {}).get('results', []) # unwrap OData envelope
df = pd.DataFrame(results)[['MeinhEq', 'Matnr', 'Ydate', 'ProdQty']]
```

Where it is stored (current state)

Files only: sap_prod_output/production_data_<from>_to_<to>.json and .csv (not DB yet).

[!] Standalone today (manual date entry, file output), not wired into the engine/DB. Credentials hardcoded in plaintext. Matnr != our skuCode format - a mapping step is needed before subtraction.

PART 2 - Our Flask API (we EXPOSE it)

Files: V1/routes/api_route.py (planning), simulation/routes/api_route.py (simulation). Registered in app.py.

How the routes are made - Flask Blueprints

```
bp = Blueprint("planning", __name__,
              url_prefix="/app/v1/jkt/planning-scheduling")
@bp.route("/plan/generate-plan", methods=["POST"])
def generate_plan(): return _generate("planning")
```

Two blueprints (planning, simulation) share one prefix; both funnel into the same _generate(mode) - only mode differs (-> jkt_* vs jkt_sim_* tables). app.py registers BOTH.

Endpoints	POST /plan/generate-plan POST /simulation/generate-plan GET /health
Request body	{ "plan_id": "<id>" } - validated: JSON object, string, non-empty, <= 50 chars
Success (200)	{ status, mode, plan_id, elapsed_seconds }
Error	{ status, stage, mode, plan_id, message } + HTTP 400/404/409/412/422/500

Orchestration inside one request

```
cfg = config_loader.load(mode=mode) # resolves jkt_* vs jkt_sim_*
plan_status.assert_not_already_scheduled() # 409 if already run
demand_route.run(cfg) # Phase A - CPS
schedule_route.run(cfg) # Phase B - LP solve (under _RUN_LOCK)
upload_route.run(cfg) # Phase C - write results
```

PART 3 - Reading INPUT from the DB

A plan_params.fetch	SELECT * FROM jkt_plan_params WHERE plan_id=%s -> dates, weights, ranks, efficiency, CO cap
A demand_db.load	SELECT * FROM jkt_demand WHERE plan_id=%s -> per-SKU demand rows
B ETL.* (6 masters)	Master_Curing_Design_CycleTime, ...Allowable_Machines_source, gt_inventory_manual, Master_WC_Master, Daily_Running_Moulds, Master_Mapping_Mould_SKU

Phase A computes CPS -> writes requirement_summary_<plan_id>.xlsx. Phase B reads that + masters, solves the LP, writes the 5-sheet PCR_Schedule_*.xlsx. All reads are %s-parameterized (no value interpolation).

PART 4 - Storing OUTPUT to the DB (Phase C)

upload_route.run(cfg) reads the schedule Excel and calls 4 writers, each targeting a table from cfg['tbl']:

kpi_writer	jkt_plan_kpis - 1 INSERT (fulfillment, SKU counts, capacity, changeovers)
plan_writer	jkt_plan - batched executemany (~14k shift rows; even-tyre rounding)
capacity_writer	jkt_plan_capacityUtilisation - one INSERT per planning day
infeasibility_writer	jkt_plan_infeasibility - CREATE IF NOT EXISTS + executemany (per at-risk SKU)

Guardrails: append-only (assert_not_already_scheduled -> 409 before any write); all writes %s-parameterized; timestamps in IST via now_ist().

Simulation mode: every jkt_* input/output table above resolves to its jkt_sim_* counterpart via cfg['tbl']; the 6 master tables are shared (not mode-aware).

The whole flow in one line

```
[SAP API -> sap_production.py -> files]    (actuals, standalone today)
Frontend -> POST /plan/generate-plan {plan_id}
-> read jkt_plan_params + jkt_demand + 6 masters (DB)
-> CPS -> LP solve -> 5-sheet Excel
-> write kpis / plan / capacityUtilisation / Infeasibility (DB)
-> 200 { status, mode, plan_id, elapsed_seconds }
Frontend reads the 4 output tables to render the report.
```

The two API surfaces are not yet connected - the SAP feed is the missing link the version2 / freezing-plan work will wire in (SAP actuals -> subtract from demand -> partial replan).